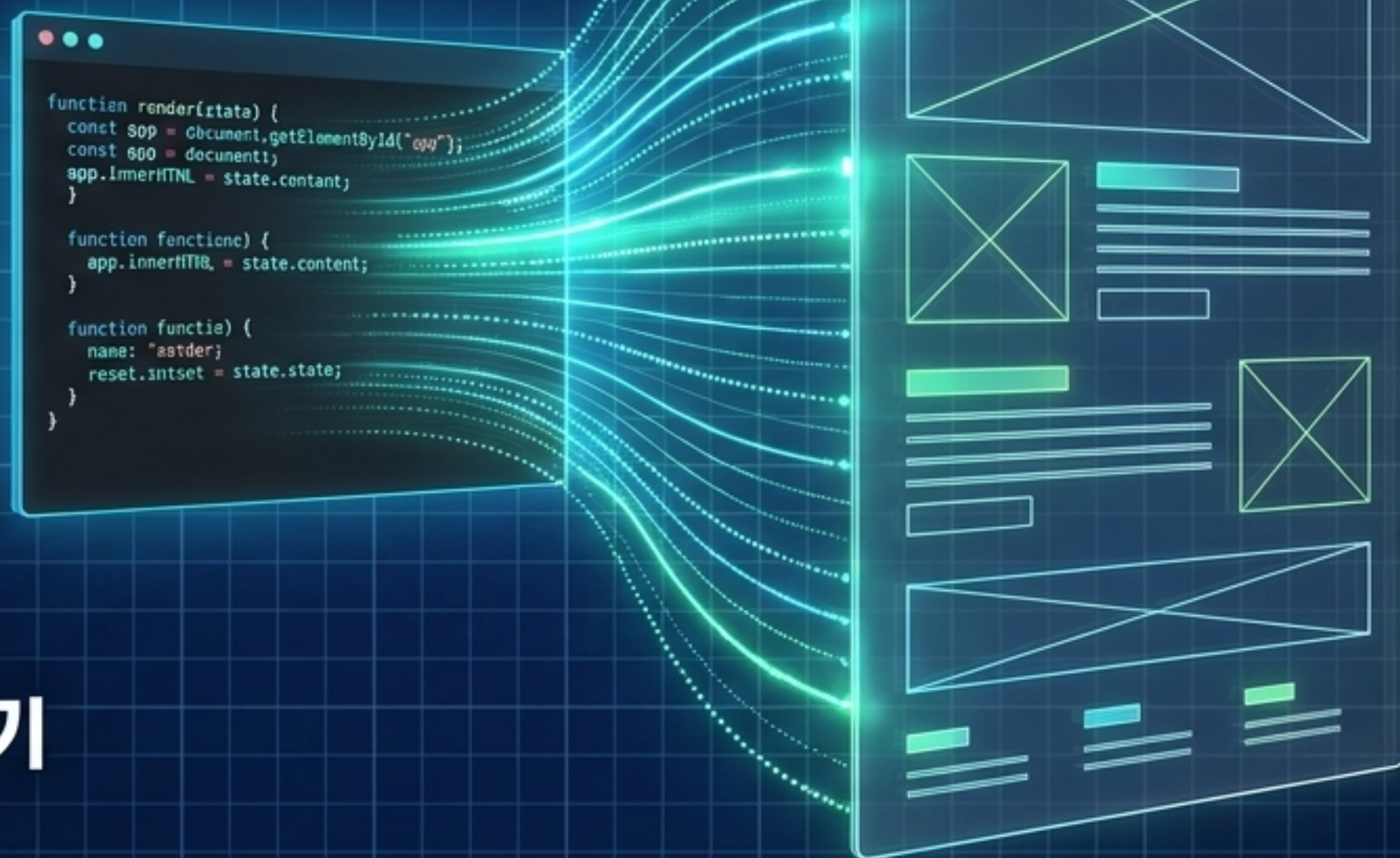
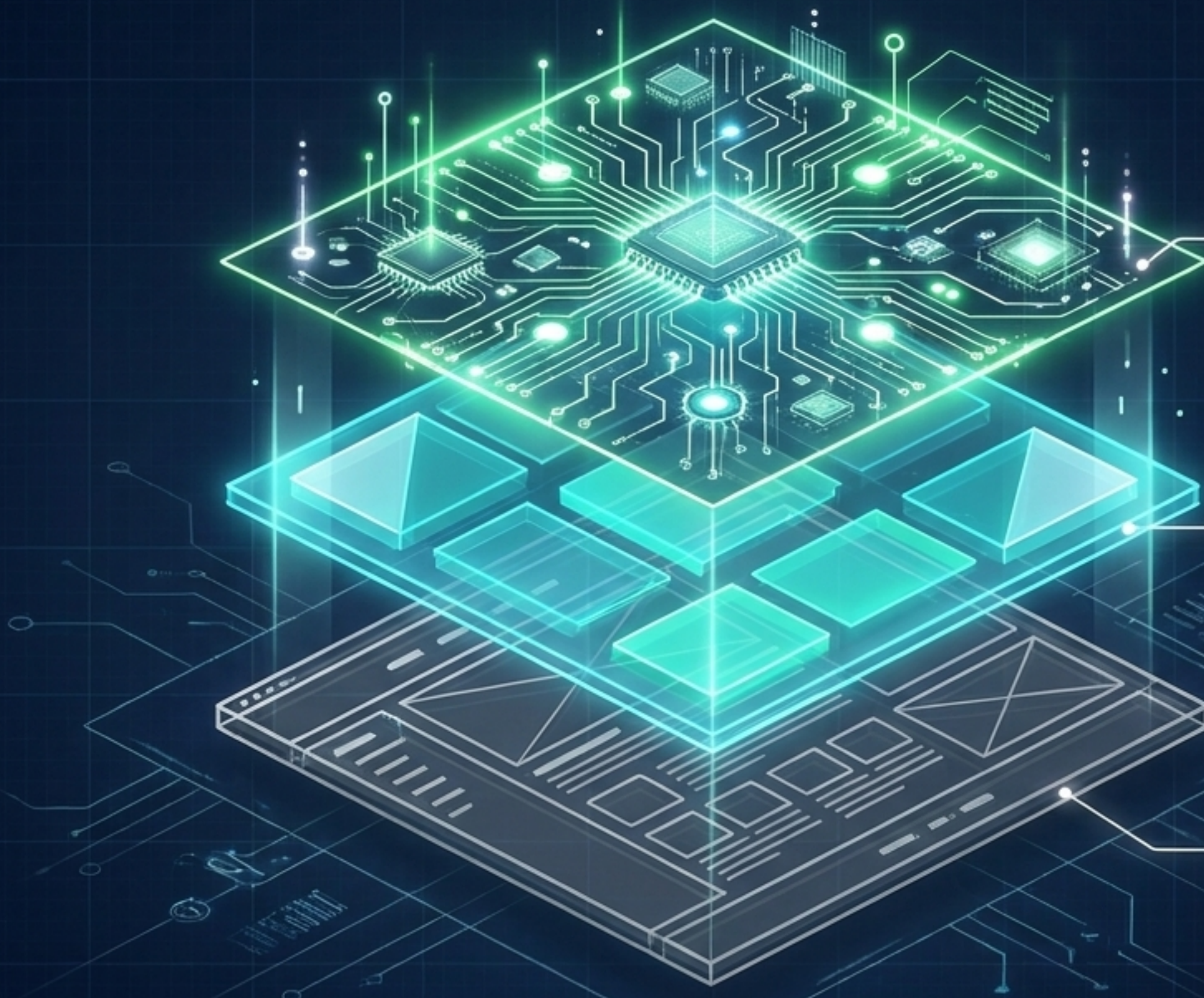


웹 아키텍처의 청사진

바닐라 JS로 구현하는
상태(State) 기반 렌더링
시스템 설계와 증명

순수 HTML/CSS/JS를 활용한
반응형 포트폴리오 웹사이트 구축기
(Codyyssey B1/B4)





JavaScript (app.js) : 전기 및 자동화.
동적 인터랙션 및 상태(State) 관리.

CSS (style.css) : 인테리어.
반응형 레이아웃 및 다크모드 테마 제어.

HTML (index.html) : 뼈대 공사.
문서의 구조와 의미(Semantic Structure) 설계.

핵심 이점: 철저한 관심사의 분리(Separation of Concerns)를 통한 코드 재사용성 극대화 및 브라우저 캐싱 성능 향상

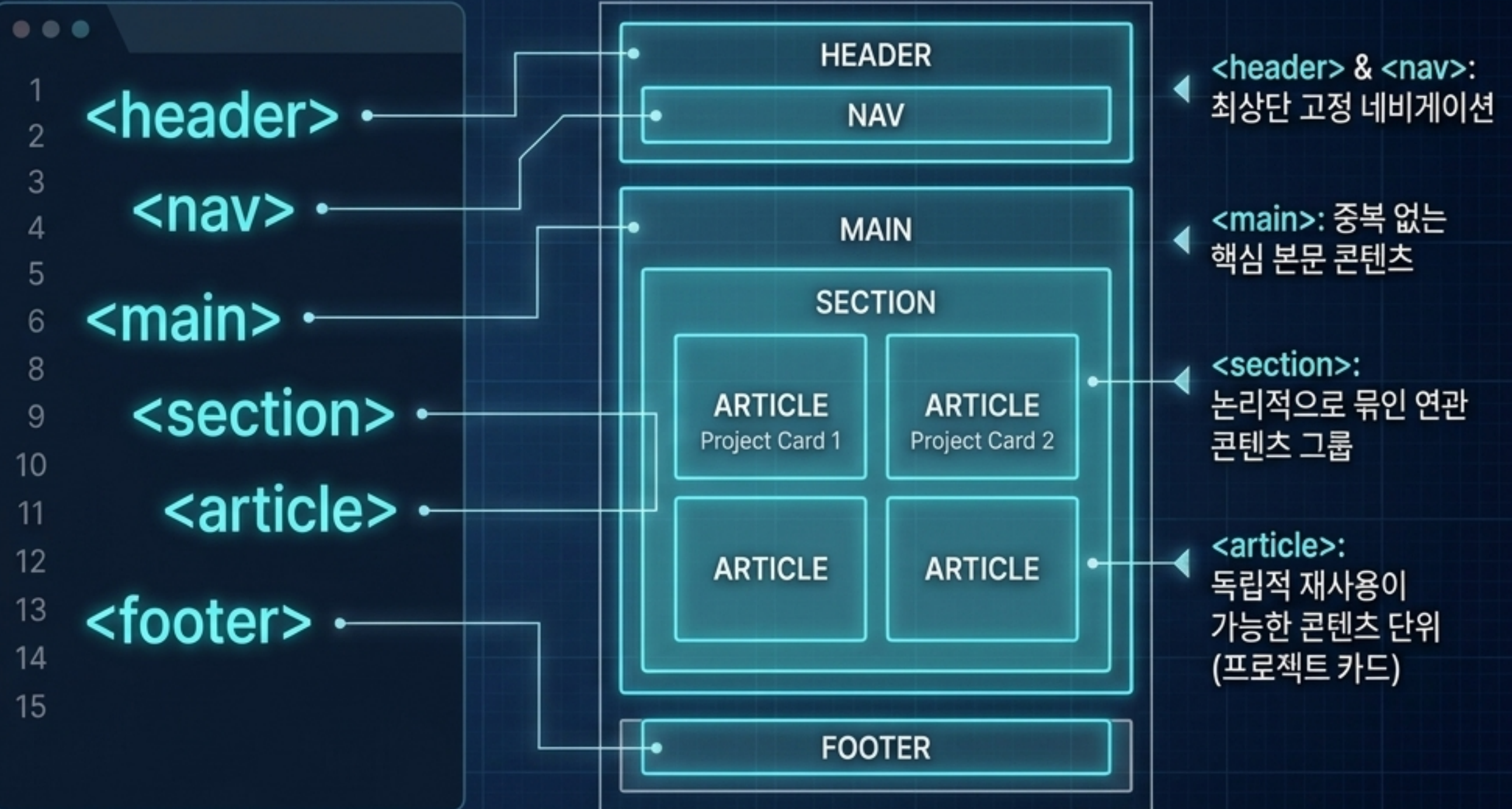
구조 설계: 왜 시맨틱(Semantic)인가?



SEO (검색 엔진 최적화):
크롤러가 핵심 콘텐츠
우선순위 색인



A11y (웹 접근성):
랜드마크 기반 직관적
탐색 지원



반응형 레이아웃 엔진: Flexbox vs Grid

Flexbox (1차원 수평/수직 제어)



Navigation 영역 적용. 로고와 메뉴 항목들을 수직 중앙, 수평 양 끝 배치

Grid (2차원 격자 제어)



Projects 카드 섹션 적용. 미디어 쿼리 없는 auto-fit 기반 자동 반응형 배치



모바일 퍼스트(Mobile First) 전략: 리소스가 제한적인 모바일 베이스를 선 작성 후, 화면이 커짐에 따라 레이아웃을 점진적으로 확장하여 불필요한 코드 최소화

디자인 시스템: 중앙 집중식 제어 (CSS Variables)

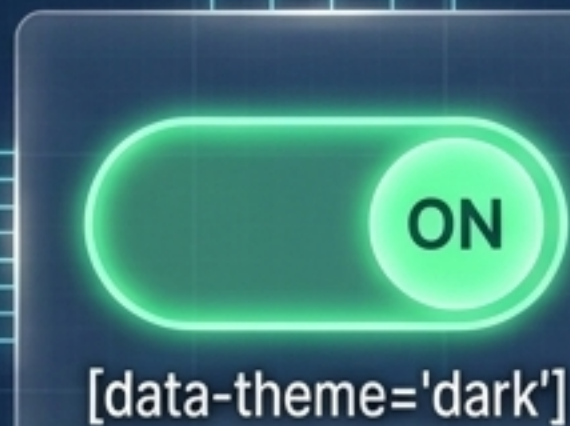
단일 제어점: 색상, 폰트 등 디자인 토큰을
:root 변수로 선언하여 일관성 유지

:root Variables

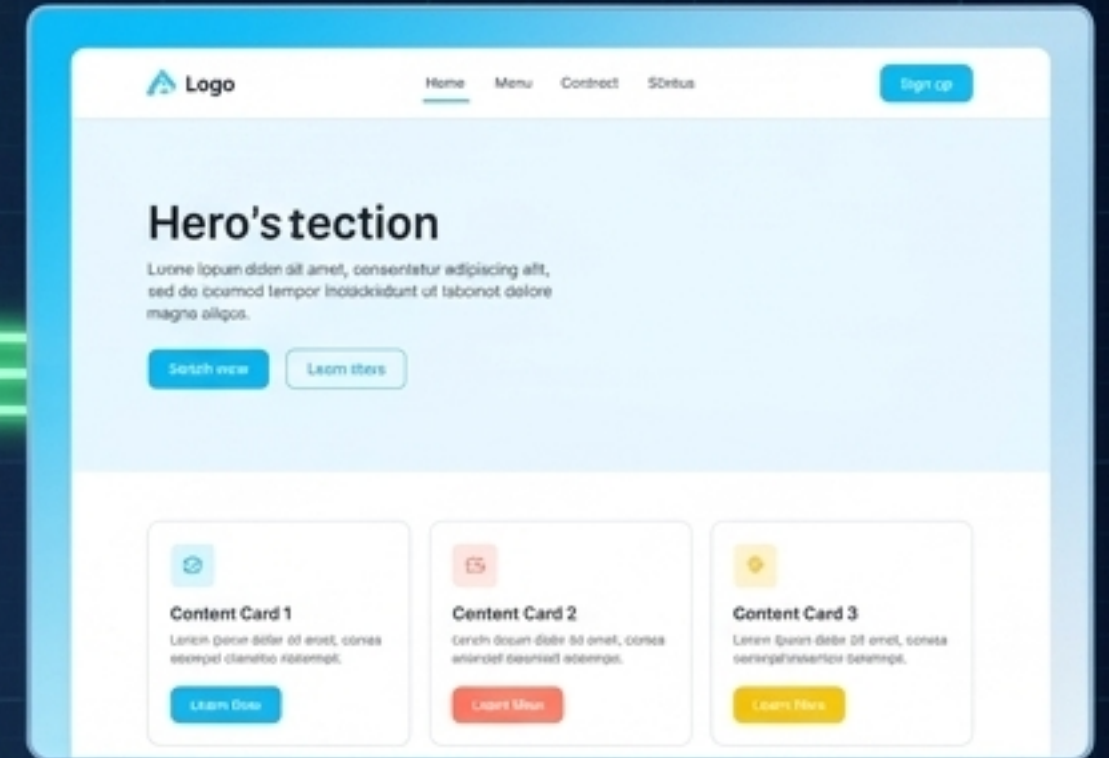
	--primary-color:		#00BFFF
	--secondary-color:		#FFD700
	--background-color:		#FFFFFF
	--text-color:		#333333

Aa	--font-stack-sans:	'Inter'
Aa	--base-font-size:	16px
Ac	--base-font-stack:	16px
Aa	--base-font-size:	16px

유지보수의 혁신: 단일 변수 값 수정으로
글로벌 업데이트 적용



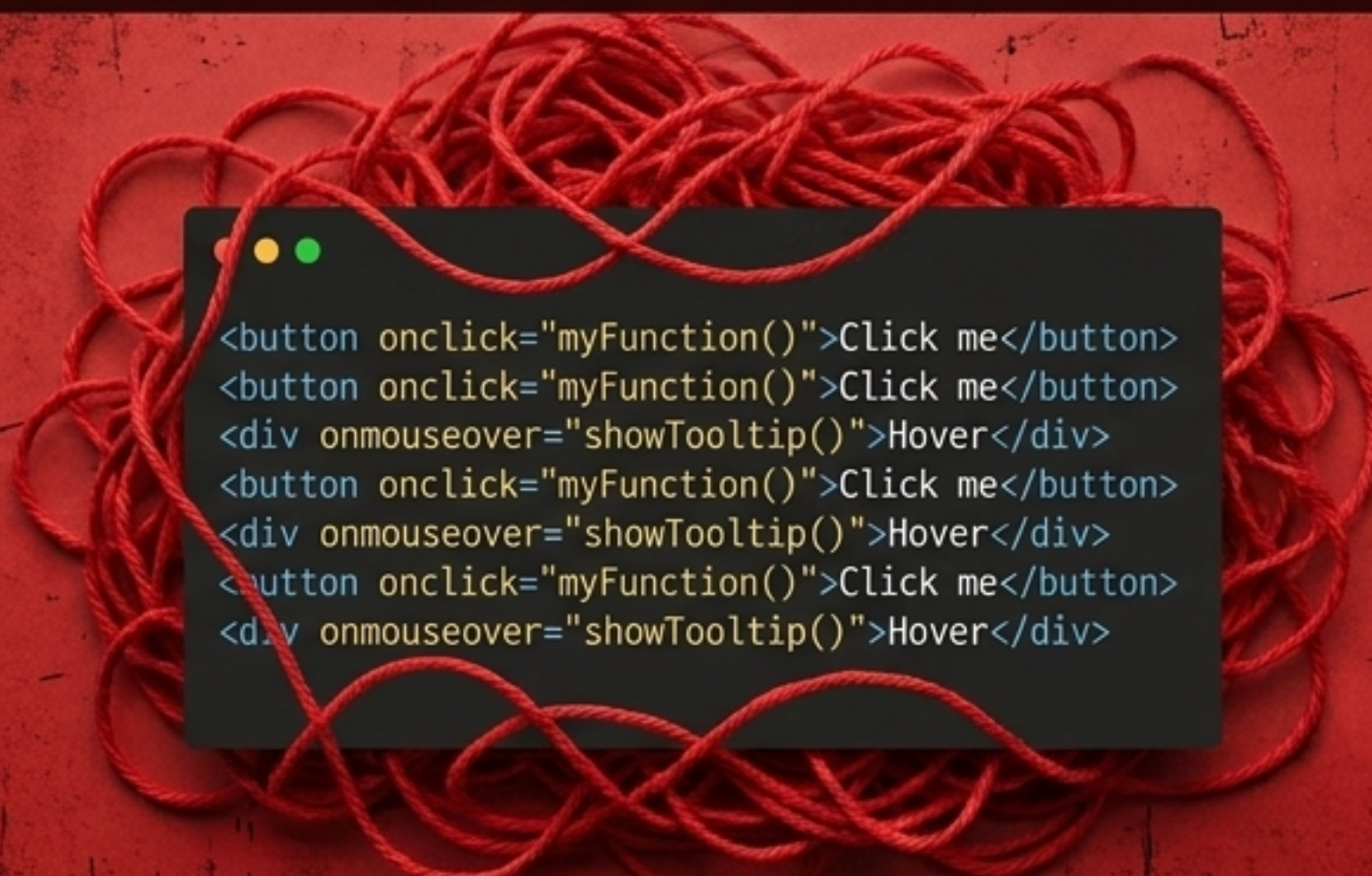
테마 전환: JS 속성 토글 하나로
전체 디자인 시스템 통째로 즉각 교체



Dark Mode

DOM 제어: 구조와 로직의 완벽한 분리

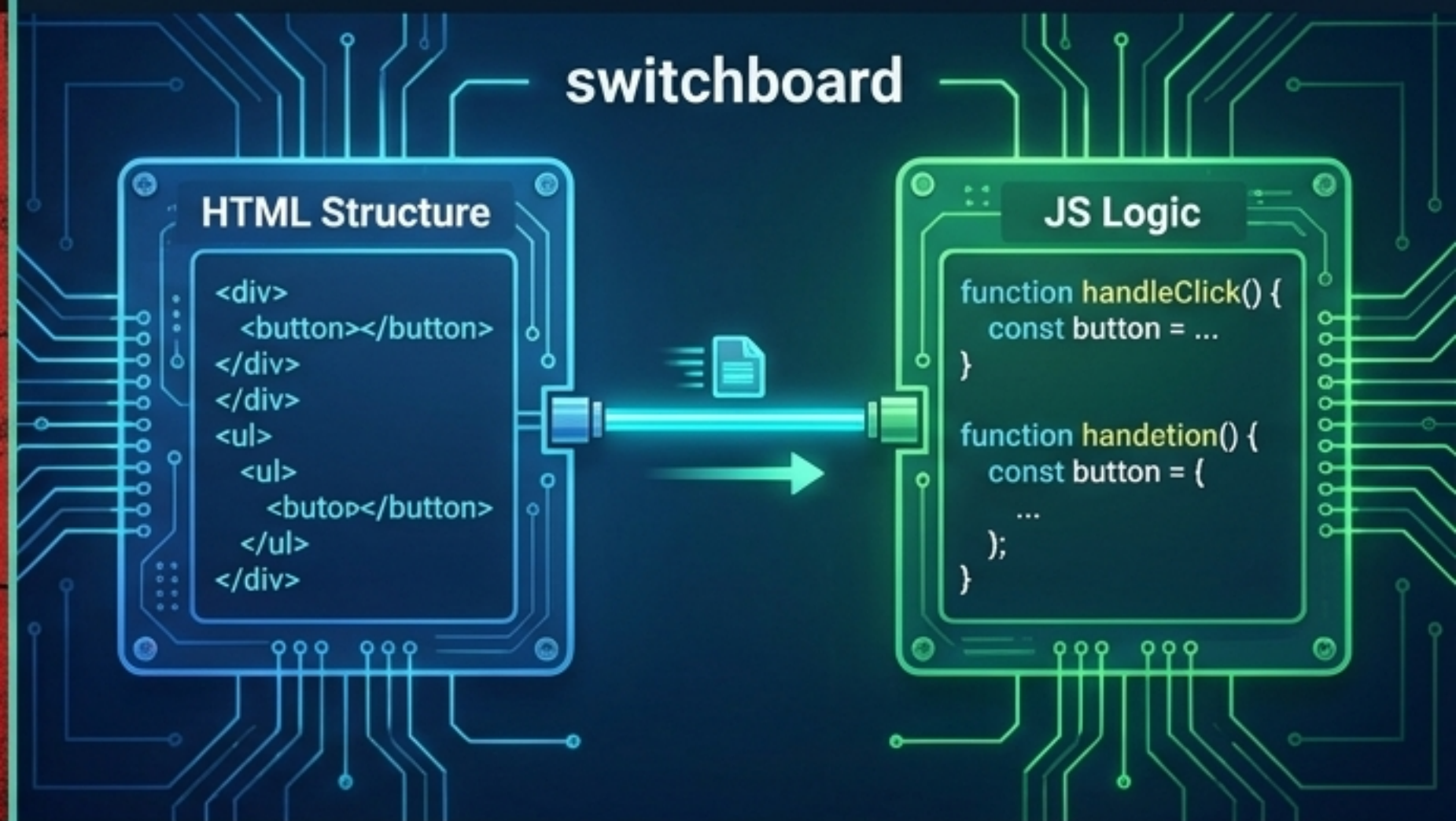
[FAIL] 인라인 onclick



```
<button onclick="myFunction()">Click me</button>
<button onclick="myFunction()">Click me</button>
<div onmouseover="showTooltip()">Hover</div>
<button onclick="myFunction()">Click me</button>
<div onmouseover="showTooltip()">Hover</div>
<button onclick="myFunction()">Click me</button>
<div onmouseover="showTooltip()">Hover</div>
```

HTML 마크업과 JS 로직의 강결합으로
가독성 저하 및 유지보수 한계

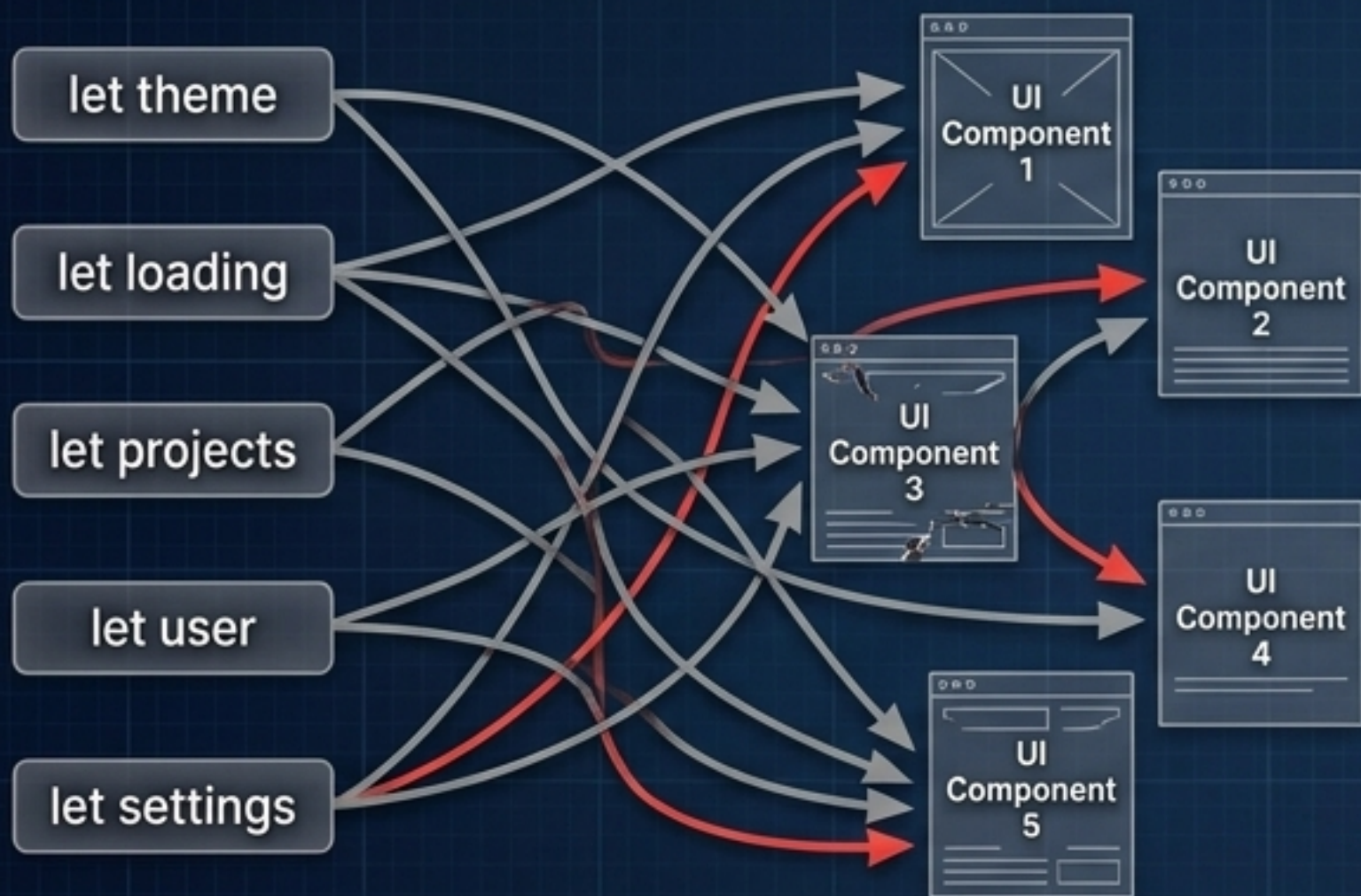
[PASS] addEventListener



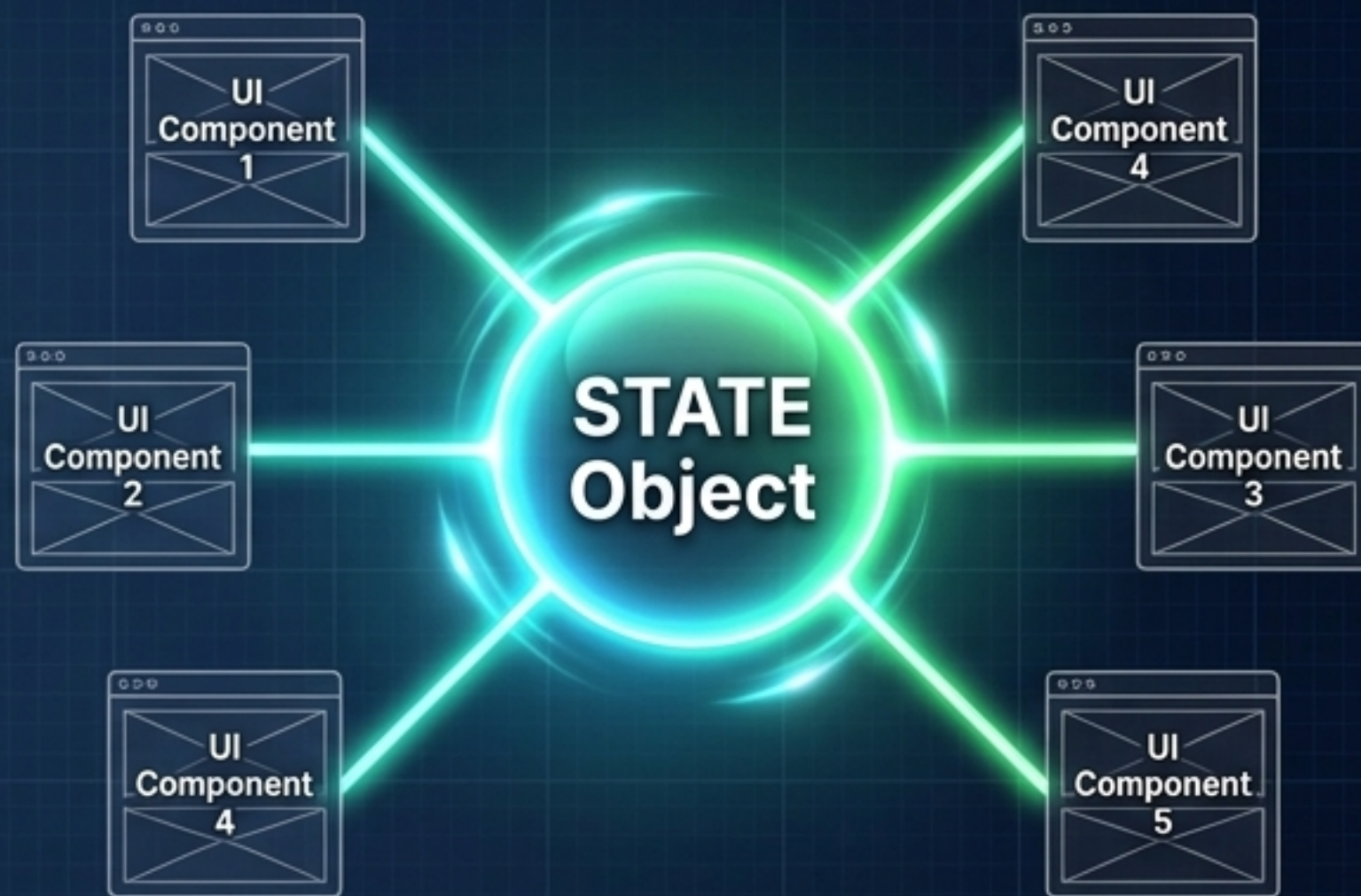
구조와 로직의 철저한 관심사 분리
동일 요소에 다중 이벤트 바인딩 지원
동적 바인딩 및 캡처링/버블링 정밀 제어

아키텍처의 심장: 통합 상태 관리 (Single Source of Truth)

문제점: 파편화된 변수로 인한 상태 추적 불가 및 사이드 이펙트 증가



해결책: 단일 출처 통제를 통해 현재 애플리케이션 상태를 한눈에 파악



엔지니어링적 의의: React의 설계 철학을 바닐라 JS 수준에서 모방하여 예측 가능한 데이터 흐름(Data Flow) 구축

핵심 렌더링 사이클: 이벤트 → 상태 → UI 업데이트



[Step 1] 사용자 이벤트 (Event)

```
button.addEventListener('click')
```

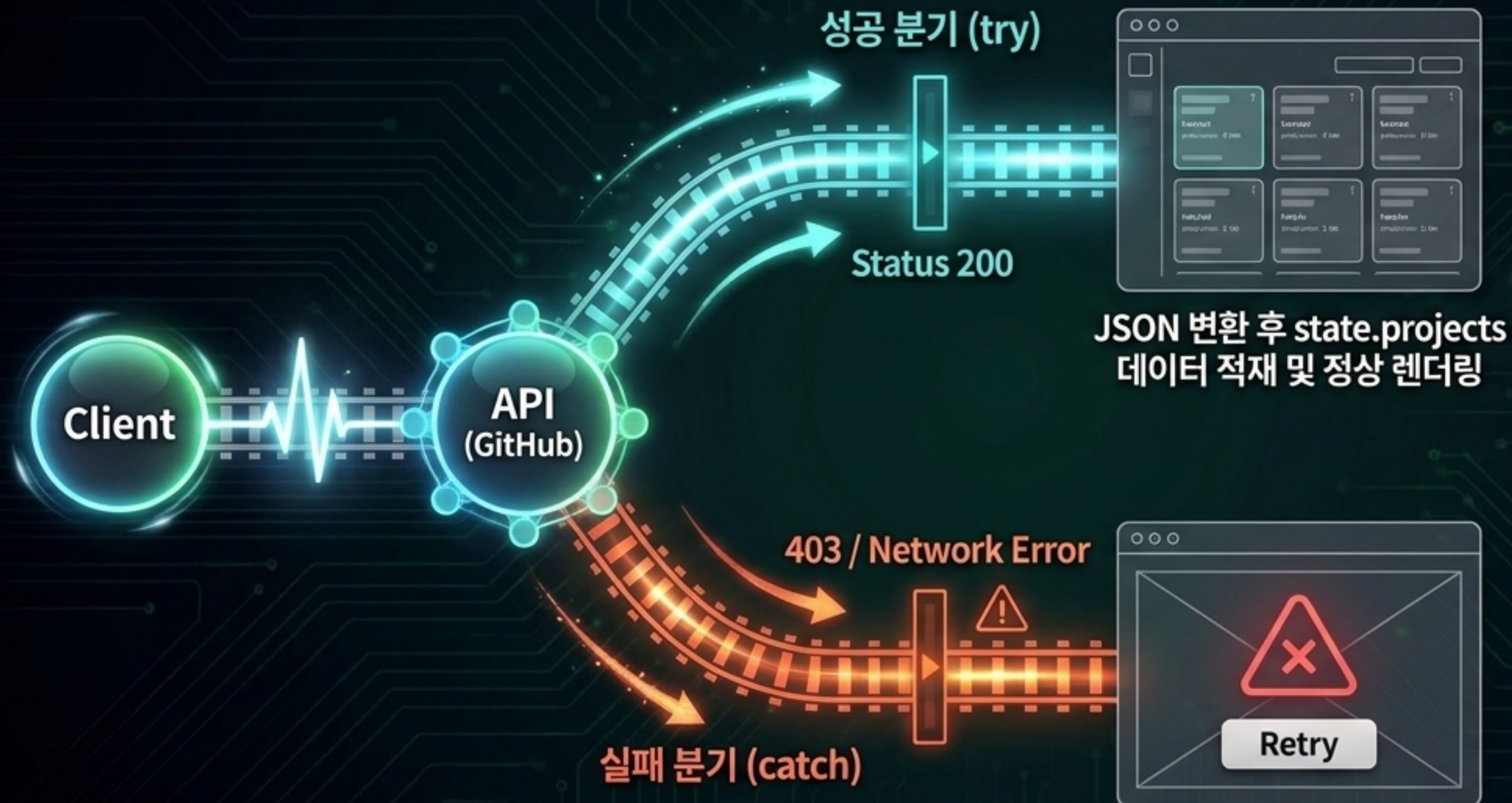
[Step 3] 화면 렌더링 (Render)

```
document.documentElement.setAttribute('data-theme', state.theme)
```

[Step 2] 상태 변경 (State Change)

```
state.theme = state.theme === 'light' ? 'dark' : 'light'
```

데이터 파이프라인: 비동기 통신과 견고한 예외 처리



JSON 변환 후 state.projects
데이터 적재 및 정상 렌더링

최신 비동기 문법:
fetch API와
async/await의 결합으로
복잡한 Promise 체이닝을
해결하고 가독성 극대화

API Rate Limit(403) 및 통신 단절 시
앱 크래시 방지 및 재시도 UX 유도

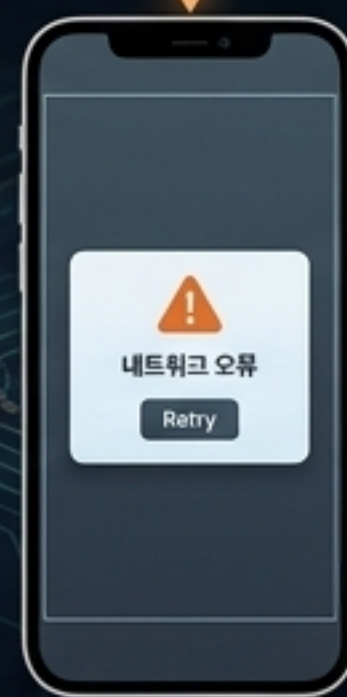
데이터 가공과 4단계 UX 상태 렌더링



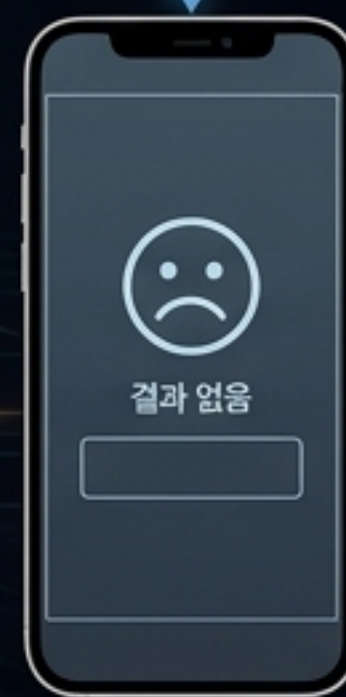
1. Loading (요청 대기)



2. Success (정상 출력)

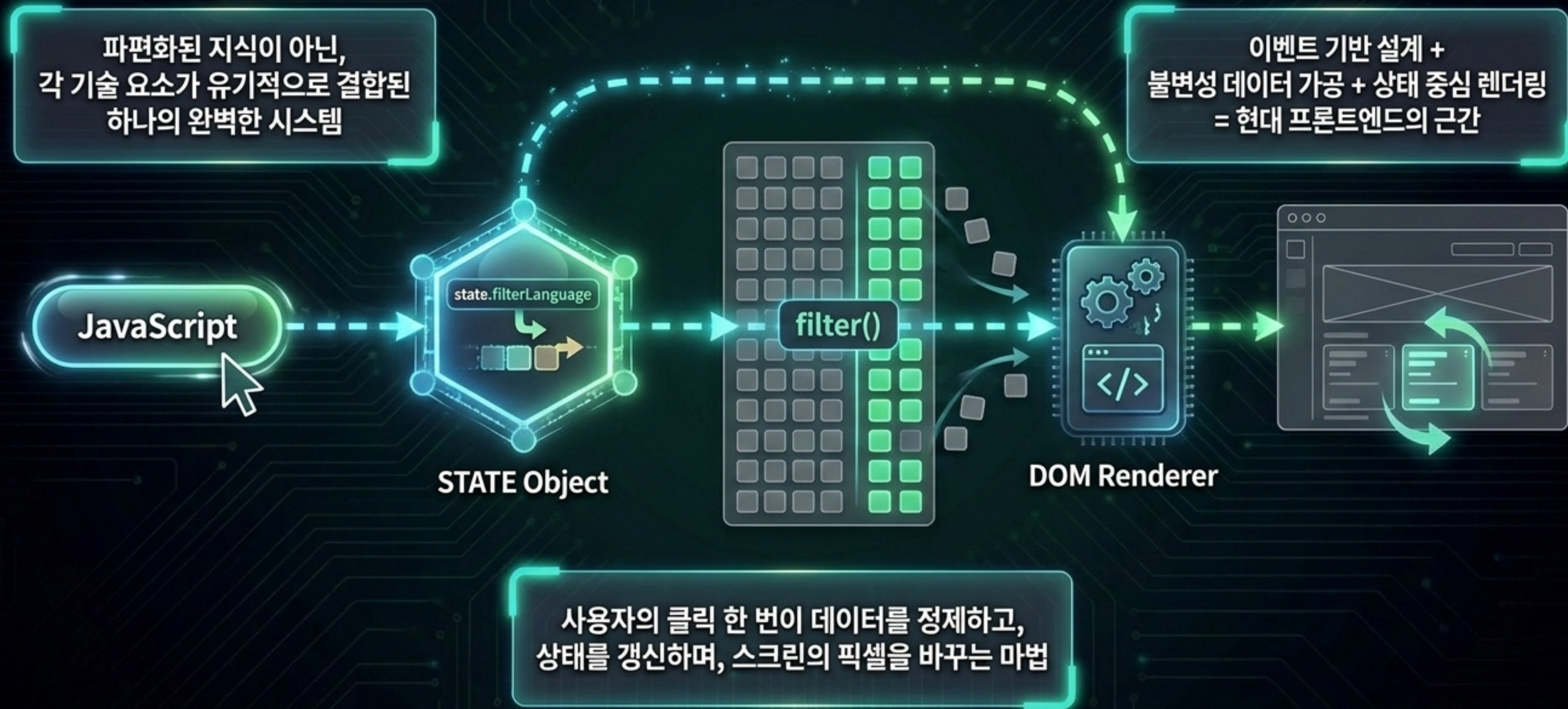


3. Error (재시도 안내)



4. Empty (결과 없음)

Synthesis: 아키텍처의 종합 작동 흐름 (The Big Picture)



Validation: 프로젝트 요구사항 100% 달성 및 고도화

PASS

[PASS] 모바일 퍼스트 반응형 설계 완수

Flexbox / Grid

PASS

[PASS] 이벤트 → 상태 → 렌더링 흐름 일관성 유지

State Object

PASS

[PASS] GitHub API 로딩/성공/실패/빈 상태 완벽 처리

try/catch

PASS

[PASS] 폼 유효성 검사 및 다크모드 유지

localStorage

[BONUS 달성] 순수 배열 메서드(filter)를 활용한
동적 프로젝트 필터링 아키텍처 구현 성공

기본기 위에 세운 프론트엔드 아키텍처

프레임워크의 편리함 뒤에 숨겨진 브라우저의 동작 원리,
DOM 제어, 비동기 처리를 바닐라 JS로 완벽히 통제하다.



Live Demo (GitHub Pages):

실제 동작하는 상태 전이와 반응형 레이아웃 확인

Source Code (GitHub Repo):

철저한 관심사 분리와 모듈화가 적용된 클린 코드

Q&A